Grok

Etiqueta de modelo: Fast
Leida: 2026-11-09T21:20:07.432Z
Caracteres: 10215
Fuentes citadas (verificables): 1
Enlaces en el DOM: 1
Fin de respuesta: inferido

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

Pregunta entendida: Opciones de interfaz (UI) y manejo de múltiples webviews nativos simultáneos, viables hoy (noviembre 2026) para un solo código base en Rust + núcleo Rust, dirigidas a Windows/macOS/Linux/Android/iOS, con prioridad absoluta a rendimiento (RAM/CPU/arranque/tamaño). Excluidas las ya evaluadas (Flutter textura/ventanas nativas, Slint + wry, Dioxus, Rune). Comparar con mediciones propias de Slint (~0,76 s arranque, 11 MiB, 74 MB RAM proceso propio, ~0 CPU extra en proceso) y Flutter; incluir fuentes fechadas, estado real por

plataforma (especialmente multi-webview nativo en Android/iOS), madurez, licencia y riesgos.
Resumen (takeaways rankeados):

Tauri 2 + wry es la opción más madura y viable para multi-webview nativos simultáneos en las 5 plataformas (incluyendo multi-window/activity/scene en Android/iOS desde ~marzo 2026). El overhead del proceso propio se acerca a Slint en desktop (casi cero CPU extra), pero el coste real lo imponen los webviews del sistema (WebView2/WKWebView/WebKitGTK/Android WebView). Binarios pequeños; arranque y RAM del proceso propio bajos; no hay mediciones publicadas de 10 webviews idénticas a las tuyas.
Freya (Skia + Dioxus-like + wry opcional) ofrece UI nativa GPU y multi-webview en desktop; Android tiene contribuciones pero webview no soportado en mobile. Rendimiento esperado cercano a frameworks de auto-dibujo (egui/iced/Slint), superior a Flutter textura.
egui/eframe e iced (immediate/retained GPU) son excelentes en desktop por bajo overhead, pero carecen de integración nativa multi-webview y soporte mobile estable/completo. No resuelven el caso de 10 webviews nativos.
GPUI (Zed) tiene composición nativa de webviews emergente en desktop y backends iOS/Android experimentales; no listo para producción multiplataforma.
Fission reclama las 5 plataformas + web con un solo código, pero es joven (estrellas bajas, releases recientes 2026); sin evidencia pública de multi-webview nativo ni benchmarks de 10 webviews.
Otras (rust_widgets, Ribir, etc.) son incompletas o demasiado inmaduras para multi-webview nativo en mobile. Ninguna opción auto-dibujada iguala el “casi cero CPU en proceso propio + webviews nativos” de Slint+wry o Tauri sin coste del motor web del sistema.
Método: Búsquedas web iterativas (frameworks Rust GUI 2025-2026, wry/Tauri multi-webview Android/iOS, benchmarks egui/iced/Slint/Freya/GPUI/Fission, estados de plataforma oficiales/PRs), apertura de docs primarias (Tauri multi-window mobile, Slint FAQs/docs, crates.io, GitHub PRs/READMEs, lib.rs). Fuentes prioritarias: docs oficiales, changelogs/PRs fechados, repos. Límites: pocas mediciones públicas de 10 webviews simultáneos idénticas a las tuyas (solo las tuyas de Windows); datos de mobile más escasos; Fission/GPUI mobile aún fluidos. Paré cuando nuevas búsquedas no alteraron conclusiones.
Hallazgos
Tauri 2 + wry (sistema webview nativo + Rust backend)
Soporta Windows (WebView2), macOS (WKWebView), Linux (WebKitGTK), Android e iOS desde Tauri 2.0 (estable ~2024, mejoras continuas 2026). Multi-webview/multi-window en mobile: PR wry#1633 y tauri#14484 merged marzo 2026 (Activity embedding en Android, UIScene en iOS); docs oficiales actualizadas mayo 2026 confirman soporte side-by-side en tablets/iPad y APIs de creación. En phones el comportamiento es stack/reemplazo, no concurrente real.
Rendimiento: proceso propio muy ligero (binarios ~3-10 MiB reportados, RAM proceso propio baja, CPU casi nula fuera de webviews); el coste lo marcan los webviews del SO (similar a tus mediciones Slint/Flutter nativos). Benchmarks independientes (2026) sitúan Tauri en rango 30-50+ MB idle (incluye helpers) y CPU superior a pure-native en dashboards vivos, pero inferior a Electron. No hay

números públicos de 10 webviews idénticos a los tuyos.
Madurez: alta (producción, ecosistema plugins, auditoría seguridad). Licencia: MIT/Apache-2.0. Riesgos: diferencias de comportamiento entre motores web del SO; permisos/capabilities; multi-window mobile requiere configuración AndroidX/scenes y no es idéntico a desktop; overhead de helpers webview en RAM total. Confianza alta en soporte de plataformas y multi-window; moderada en rendimiento exacto de 10 webviews (falta medición propia).GitHub +5
Freya (Skia/GPU + modelo componentes estilo Dioxus, wry opcional)
Desktop (Windows/macOS/Linux) maduro; Android con contribuciones (freya-android); web experimental (WASM/canvas). WebView nativo vía freya-webview (wry): solo desktop (WebView2/WKWebView/WebKitGTK X11); Android/iOS/Wayland explícitamente no soportados. Multi-webview posible en desktop como elementos de layout.
Rendimiento: auto-dibujo Skia; benchmarks de ecosistema 2026 lo sitúan en rango competitive de iced/egui/Slint (binarios ~18 MiB, CPU/RAM bajos en dashboards). Esperable superior a Flutter textura y cercano a tus Slint nativos en proceso propio.
Madurez: media-alta en desktop (releases 0.4/0.5 2026, multi-window, HTML via Blitz). Licencia: MIT. Riesgos: webview mobile ausente; Android incompleto; cambios de API. Confianza moderada en desktop; baja en mobile completo.Lib +2
egui + eframe (immediate-mode GPU)
Desktop + web (WASM) sólidos; Android soportado (eframe 0.30+ 2024/2025, winit/android-activity); iOS con soporte reportado pero menos maduro. No hay integración nativa de webviews multi (solo overlays HTML en web o hacks).
Rendimiento: excelente (arranque <100-300 ms, binarios 4-10 MiB, RAM/CPU bajos en benchmarks 2026; a menudo mejor o comparable a Slint en idle/simple). Superior a Flutter.
Madurez: alta en desktop/herramientas. Licencia: MIT/Apache. Riesgos: no resuelve multi-webview nativo; UI “no nativa”; mobile aún con limitaciones de input/IME/lifecycle. Confianza alta en desktop performance; baja para el caso multi-webview.Crates +1
iced (Elm/MVU + wgpu)
Desktop + web; mobile experimental (Android vía winit/wgpu + integration, iOS similar pero no oficial). Sin webview nativo multi.
Rendimiento: bueno (binarios ~8 MiB, CPU/RAM competitivos con egui/Slint en mediciones 2026).
Madurez: alta desktop (0.14 2025/2026, usado en COSMIC). Licencia: MIT. Riesgos: mobile no prioritario; widgets limitados vs nativos. Confianza moderada-alta desktop; baja mobile.GitHub +1
GPUI (Zed, GPU hybrid)
Desktop (macOS prioritario, Windows/Linux mejorando); iOS backend experimental (PRs 2025-2026); Android community ports. Composición nativa de webviews (wry/WKWebView/WebView2) emergente 2026 (overlays, z-order).
Rendimiento: muy alto en desktop (binarios pequeños ~4 MiB, CPU bajo).
Madurez: media (pre-1.0, breaking changes, ecosistema cerrado). Licencia: ver repo Zed. Riesgos: mobile no producción; webview composition aún en evolución; documentación limitada. Confianza

baja para 5 plataformas estables.GitHub +2
Fission
Reclama macOS/Windows/Linux/Android/iOS/Web + terminal/SSR con un solo código (Vello/wgpu, widgets declarativos). Releases 2026 (0.15), CLI de packaging. Sin evidencia pública de multi-webview nativo ni benchmarks de 10 instancias.
Rendimiento: desconocido medido (GPU-accelerated, se espera competitivo).
Madurez: baja-media (pocas estrellas, joven). Licencia: Apache-2.0. Riesgos: inmadurez, falta de validación multi-webview/mobile real, cambios API. Confianza baja.Fission
Otras (rust_widgets, Ribir, WaterUI, Qt Bridge, etc.)
rust_widgets reclama mobile (iOS/Android) + desktop con auto-dibujo; webview group mencionado pero no multi-nativo validado. Ribir desktop/web, mobile 🚧. Qt Bridge solo desktop. Ninguna aporta multi-webview nativo verificado en Android/iOS + rendimiento medido comparable.Lib +1
Comparación de rendimiento (medido o reportado vs tus datos):

Proceso propio / arranque / tamaño: Slint y pure-Rust (egui/iced/Freya/GPUI) lideran (~0,5-1 s, <15 MiB, <100 MB). Tauri similar en proceso propio pero RAM total sube por webviews. Flutter textura añade CPU extra (tus 5 puntos).
Con 10 webviews nativos: el cuello de botella es el motor del SO en todos los casos (Tauri/Slint+wry/Freya-desktop). No hay datos públicos que superen tus mediciones de Slint nativo en Windows. Benchmarks generales 2026 confirman pure-native ~79-110 MiB vs webview helpers 200+ MiB.Goldstrikearch.github +1
Fuentes (selección)

Fuente | Tier | Fecha | Tauri docs multi-window mobile, wry/tauri PRs #1633/#14484 | 1 (oficial) | 2026-03 a 2026-05 |
Slint FAQs/docs mobile/Android/iOS | 1 | 2025-2026 |
Freya docs/crates freya-webview, releases 0.4/0.5 | 1 | 2026 |
egui/eframe changelog, Android PRs | 1 | 2024-2026 |
goldstrikearch Rust GUI ecosystem measured | 2 | 2026-09 |
crates.io / lib.rs (wry, freya, fission, iced, etc.) | 1 | 2026 |
GPUI/Zed PRs composition/webview/iOS | 1 | 2026 |
Fission.rs / crates | 1 | 2026-10 |
Conflictos y gaps:

Multi-webview mobile real solo documentado de forma sólida en Tauri (Activity/Scene); el resto es desktop o experimental.
Benchmarks de 10 webviews simultáneos son casi exclusivos de tus mediciones Windows; no hay réplicas públicas cross-platform.
Fission y GPUI mobile cambian rápido; claims de “todas las plataformas” superan evidencia de uso real.
Licencias comerciales (Slint) vs permissivas (Tauri/egui) no comparadas en profundidad aquí.

No se midió en este análisis el overhead exacto de DOM reading/file attach en cada opción.
Preguntas abiertas / próximos pasos:

Medir Tauri 2 con 10 webviews nativos en la misma máquina Windows (y luego Android/iOS tablet) usando el mismo workload de redibujado/DOM/lectura.
Prototipo mínimo Freya + wry multi-webview desktop + prueba Android (sin webview).
Confirmar en dispositivos reales el comportamiento de Activity embedding / UIScene con 10 instancias (¿límites de memoria del SO?).
Evaluar si el “solo código” de Fission incluye realmente multi-webview nativo o requiere capas adicionales.
Coste de falsificación: si el multi-webview mobile de Tauri no escala a 10 en phones/tablets de gama media, el plan de un solo código se rompe y habría que bifurcar mobile.
Confianza global: alta en el ranking de viabilidad actual (Tauri lidera el caso multi-webview 5 plataformas); moderada en números de rendimiento exactos fuera de desktop (faltan mediciones propias reproducibles).

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.
Este documento es la respuesta a la pregunta, no la evaluacion que este
proveedor hizo de las respuestas de los demas.